AI Integration Report

Week 08 — AI Integration & Intelligent Features Development

GEMINI · HUGGINGFACE

Course	CSE 4104 — Software Development III
Team	CSE4104-7C-T05
Project Title	AI Academic Mentor
Instructor	Md. Riaz Mahmud, Assistant Professor, Dept. of CSE, NUBTK
Section	7C
Submission Date	13 August 2026
GitHub Repository Link	GitHub - Priyo-1158/cse4104-7c-t05-ai-academic-mentor: AI Academic Mentor - Personalized Learning Assistant for Students · GitHub

Team Information

Role	Member Name	Student ID	Contribution
Team Leader	Shadman Sadequeen Priyo	11230121158	Project coordination, GitHub management, integration testing
Frontend Developer	Musabbir Hossain Chayon	11230121168	API integration, Report, UIpages, Netlify deployment,Next.js
Backend Developer	Sk. Nadirul Haque Nadir	11230121155	Node.js, Express, routing, Express routes, MongoDB models, JWT authentication
AI Integration Lead	Shafin Kabir	11230121175	Gemini API, HuggingFace, AI routes

TABLE OF CONTENTS

1. AI Feature Integration

Four AI-powered features are wired end-to-end from frontend UI through the backend proxy to the AI provider chain, each solving a distinct part of the "academic mentor" concept rather than existing only to demonstrate an API call. This directly satisfies the assignment's core requirement — "at least one meaningful AI-powered feature" — several times over.

FEATURE / ENDPOINT	WHAT IT DOES & WHY IT MATTERS	PERSISTED OUTPUT
AI Chatbot `POST /api/ai/chat`	Free-form academic Q&A for concepts students get stuck on outside class hours. Suggested prompts (recursion, Big-O, normalization, JWT, OSI model) seed first-time users.	Note document (kind: chat) written per exchange when logged in.
Quiz Generator `POST /api/ai/quiz`	Generates N multiple-choice questions on a chosen topic at a chosen difficulty, turning passive reading into active self-testing.	Score, correct count, per-question answers via POST /api/quiz.
Note Summarizer `POST /api/ai/summarize`	Condenses pasted notes into concise/detailed/bullet/exam-style summaries — addresses reading-volume bottleneck before exams.	Saved via POST /api/summary as a Note (kind: summary).
Study Planner `POST /api/ai/plan`	Builds a multi-day, hours-per-day revision schedule across chosen subjects, replacing ad-hoc cramming.	Saved via POST /api/plan as a StudyPlan document.

2. Prompt Engineering

Each feature is driven by a dedicated system prompt that constrains the model to a specific output shape, combined with a dynamically built user prompt from the student's actual input. A design rationale is included for each, since the brief asks teams to explain their prompt-design strategy, not just show the prompt text.

2.1 — AI Chatbot

System Prompt	`(none currently set — general-purpose academic assistant)`
User Prompt	`Explain recursion using a simple real-life example.`
Expected Output	A short, beginner-friendly conceptual explanation returned as plain markdown-style text and rendered in the chat window.

2.2 — Quiz Generator

System Prompt	`Act as a quiz generator that outputs strict JSON only.`
User Prompt	`Create 5 multiple-choice questions on "Database Management System" at medium difficulty. Return ONLY valid JSON: {"questions":[{"question":"...","options":["A","B","C","D"],"answer":"..."}]}. No markdown, no extra text.`
Expected Output	A JSON array of 5 question objects, each with 4 options and a correct answer, parsed by tryParseJSON() and rendered as an interactive quiz.

2.3 — Note Summarizer

System Prompt	`(none — the instruction is inlined into the user prompt)`
User Prompt	`Summarize this text in a concise style: [pasted student notes]`
Expected Output	A shortened paragraph preserving key facts, trimmed of whitespace and stored against the note's subject/style.

2.4 — Study Planner

System Prompt	`Act as a study planner that outputs strict JSON only.`
User Prompt	`Create a 5-day study plan (3 hours/day) covering: Data Structures, DBMS. Return ONLY valid JSON: {"days":[{"day":1,"sessions":[{"subject":"...","topic":"...","duration_min":60, "type":"lecture"}]}]}. No markdown.`
Expected Output	A JSON array of day objects, each with timed study sessions per subject, parsed and rendered as a calendar-style plan.

3. AI Workflow Design

Every AI feature follows the same seven-stage pipeline, shown below with the actual file each stage lives in. The frontend never talks to an AI provider directly — all calls are proxied through the Express backend so provider keys never reach the browser bundle.

1	**User Input** Student types a question, pastes notes, or sets quiz/plan parameters in a React page (Chatbot.jsx, Quiz.jsx, Summarizer.jsx, Planner.jsx).
2	**Frontend Proxy Call** aiApi.js (the GeminiAPI object) posts JSON to the backend — e.g. POST /api/ai/chat — never holding an AI key client-side.
3	**Backend Route + Soft Auth** The Express route in routes/ai.js runs softAuth middleware to attach req.user if a valid JWT is present, without blocking guest access.
4	**Prompt Assembly** processAIChat() combines a feature-specific system prompt with the user's input into one prompt string.
5	**Provider Chain** Gemini (gemini-2.5-flash) is tried first; on failure, HuggingFace (Qwen2.5-7B-Instruct); if both fail, a static template — the endpoint always responds 200.
6	**Response Validation** tryParseJSON() extracts/parses JSON for quiz/plan; chat/summary text is trimmed. The active source is tagged on every response.
7	**Persist + Display** Logged-in chat exchanges are written to MongoDB immediately; quiz/plan/summary results persist when the student saves them.

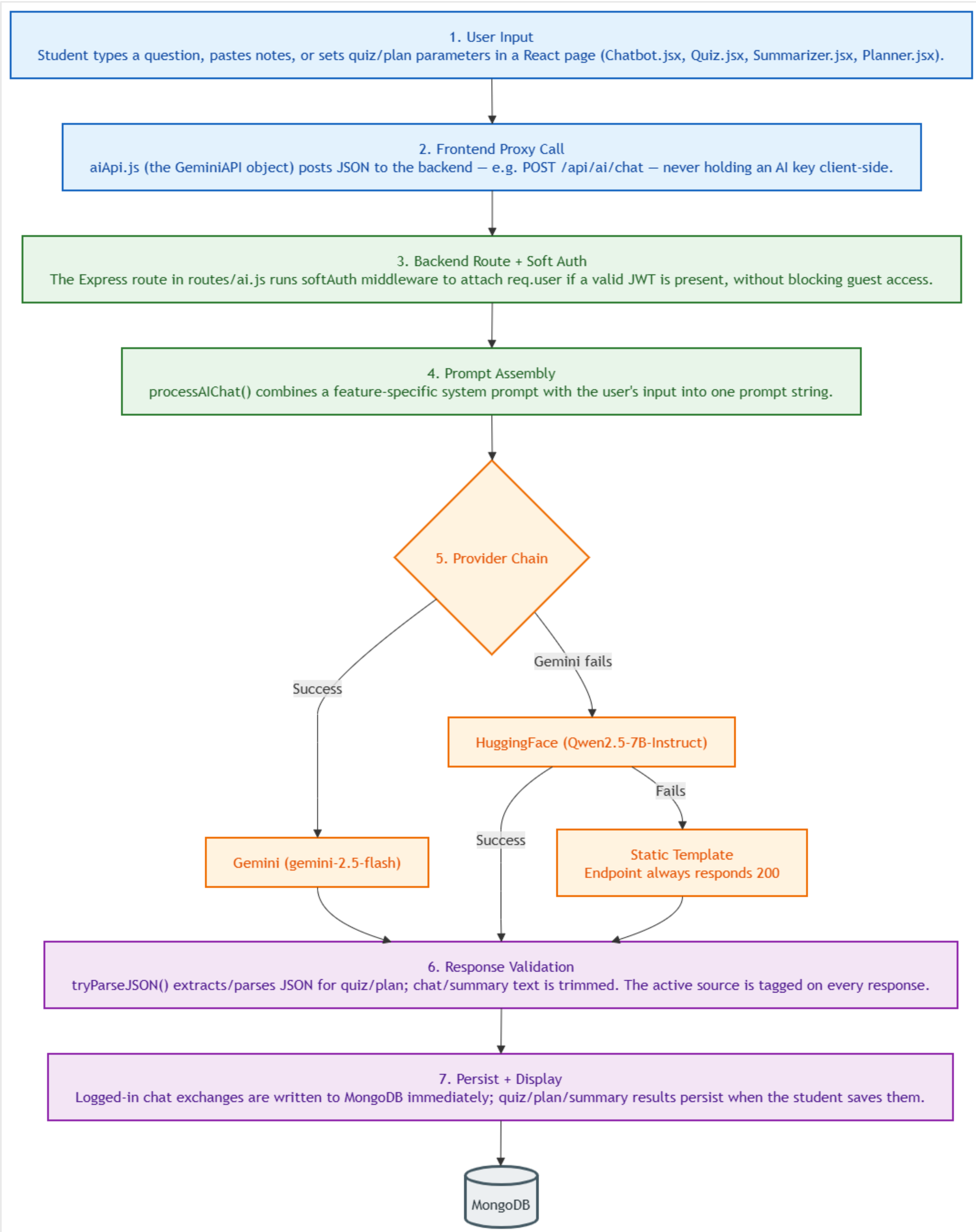


Figure : AI Workflow Design

4. AI Response Handling

The assignment brief requires explicit handling for six network/provider scenarios. The matrix below states, for each one, exactly what the code does today and whether that satisfies the requirement — including the gaps that are not yet closed.

SCENARIO	CURRENT BEHAVIOUR	STATUS
Successful response	Response tagged with its source ("Live Gemini AI API" / "HuggingFace Chat" / "Static Fallback") so the true origin is always visible.	Implemented
API errors (provider exception)	Each provider call is wrapped in its own try/catch; a Gemini exception falls through to HuggingFace; a HuggingFace exception falls through to the static template.	Implemented
Network failure	Covered by the same try/catch + fallback chain as API errors — a network-layer exception is caught identically to an API-layer one.	Implemented
Rate limits	A general 100-req/15-min limiter covers /api/ (including AI routes). A dedicated aiLimiter (60/min) exists in server.js but is never mounted.	Partial — wire up aiLimiter
Timeout scenarios	No explicit timeout is set around the Gemini or HuggingFace SDK calls; a hung provider blocks on the SDK's own default timeout.	Not implemented

5. AI Output Validation

The assignment separates output validation from response handling: this section is about what happens to a reply that technically arrived, before it reaches the student.

REQUIREMENT	IMPLEMENTATION	STATUS
Check for empty responses	processAIChat() checks response.text as both a callable and a plain field, falls back through response.candidates[0].content.parts[0].text, and finally to the literal string "No response generated."	Implemented
Remove unwanted formatting	tryParseJSON() regex-extracts the first {...} or [...] block, stripping stray markdown fences for quiz/plan. Chat/summary text only gets .trim() — no markdown/HTML stripping.	Partial
Prevent duplicate outputs	No dedup check exists in any AI route — two identical requests in a row will generate and persist two separate results.	Not implemented
Display meaningful error messages	Validation failures return specific text (e.g. "topic, score, and numQuestions are required.") and expired-token failures return "Token has expired. Please login again." rather than a generic 400/401.	Implemented

6. AI Usage Documentation

6.1 — Why AI Is Required

The application's core proposition is a "mentor" that explains concepts on demand, drills students with generated questions, condenses arbitrary pasted notes, and builds a personalized revision schedule. None of these four tasks can be delivered with static or rule-based logic: each requires natural-language generation conditioned on unpredictable student input.

6.2 — Selected AI Platform & Model

Google Gemini is the primary platform, with HuggingFace as an automatic secondary provider and a static local responder as a last-resort tertiary fallback — no single outage or malformed key can take the AI features fully offline.

TIER 1 — PRIMARY **Google Gemini**
gemini-2.5-flash · `@google/genai SDK`
Strong free-tier limits and low latency, matching the "flash" class recommended for interactive use.

TIER 2 — FALLBACK **HuggingFace Inference**
Qwen2.5-7B-Instruct · `@huggingface/inference SDK`
Open-weight model used only when Gemini throws — keeps the feature live without a second paid dependency.

TIER 3 — LAST RESORT **Local static responder**
Rule-based templates · `In-process JS, no network call`
Guarantees HTTP 200 with readable content even if both network providers are unreachable.

6.3 — Prompt Design Strategy

Summarized in full in §2: structured-output features (quiz, plan) use a strict "JSON only, no markdown" system prompt to guarantee machine-parseable results; free-text features (chat, summary) favor an open or inlined instruction to preserve natural tone.

6.4 — Expected Outputs

FEATURE	EXPECTED OUTPUT
AI Chatbot	**Beginner-friendly conceptual explanation as plain text.**
Quiz Generator	**5-question MCQ set as strict JSON, each with 4 options + answer.**
Note Summarizer	**Condensed paragraph matching the requested style.**
Study Planner	**Multi-day session schedule as strict JSON.**

6.5 — Current Limitations

- Primary provider (Gemini) is currently unreachable in practice due to the malformed API key — see §1. All live demos today run on Tier 2/3.
- No explicit request timeout on either AI SDK call (§4).
- The AI-specific rate limiter (aiLimiter) is defined but not attached to any route (§4).
- No duplicate-output prevention on any AI endpoint (§5).
- Chatbot has no system-prompt persona, so tone/scope consistency relies on the base model's defaults (§2.1).
- Admin "Announcements" and "Bulk Clear / Wipe Data" panels are intentional non-functional stubs per the project README.

6.6 — Future Improvements

- Replace the Gemini key with a valid aistudio.google.com Developer API key and re-verify a live end-to-end call before final submission.
- Add an AbortController-based timeout (e.g. 10 seconds) around both provider calls in processAIChat().
- Mount the existing aiLimiter middleware on the /api/ai router.
- Add a lightweight duplicate-request guard and a named chatbot persona.
- Surface the response source tag (Gemini / HuggingFace / Static) in the UI itself, not just the API payload.

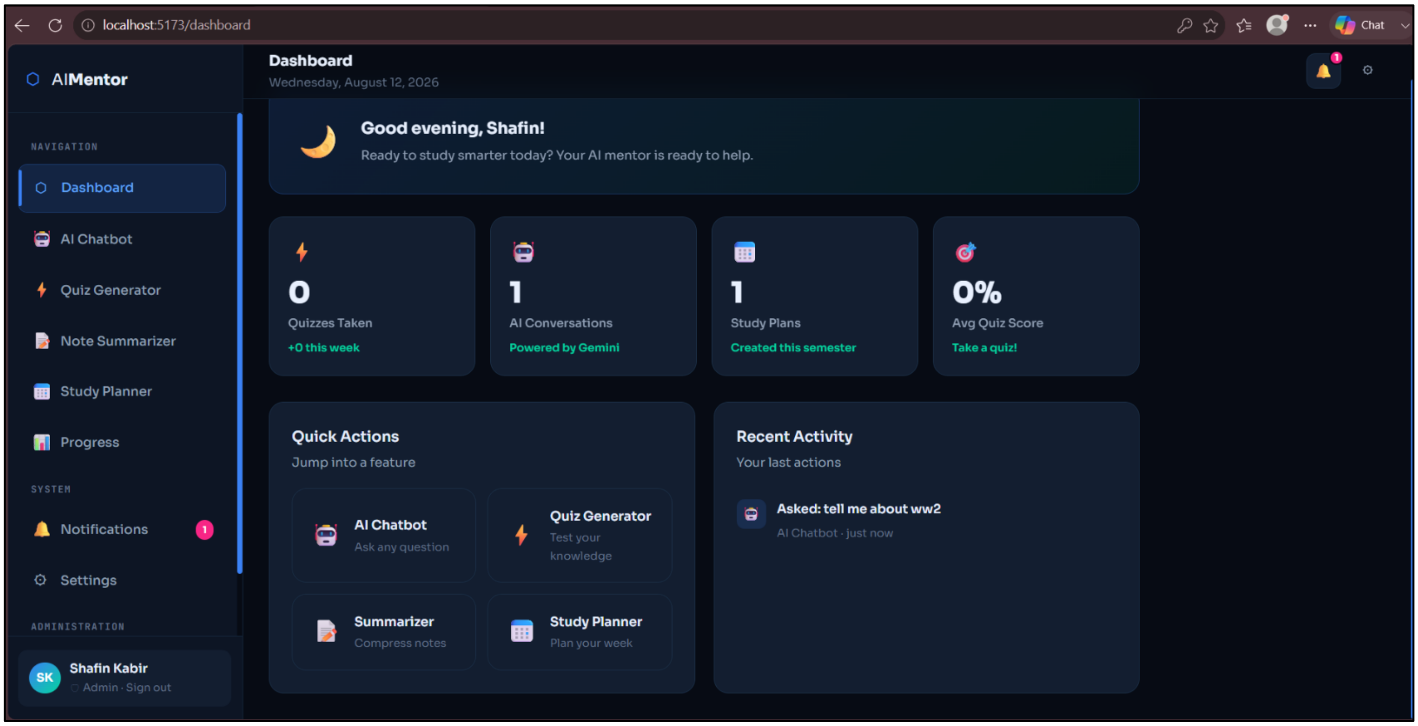
7. Current Development Progress

Distinct from §6.5's limitations: this section states build status per component, reflecting the actual state of the codebase in this submission.

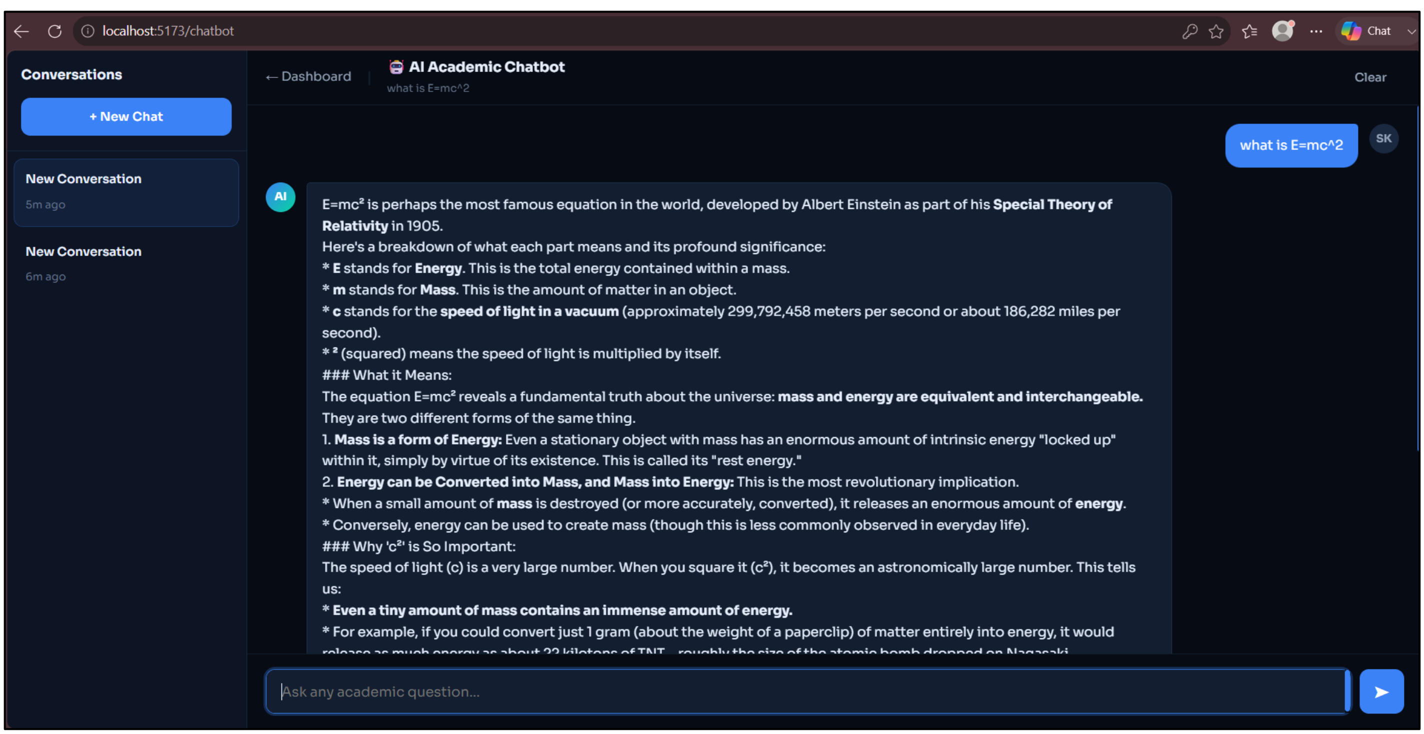
COMPONENT	STATUS	NOTES
Auth & DB models	Complete	JWT auth, bcrypt hashing, User/Note/QuizResult/StudyPlan Mongoose models in active use.
AI Chatbot / Quiz / Summarizer / Planner	Functional (fallback tier)	End-to-end wired; currently answering via Gemini /static fallback, not HuggingFace
Progress Tracking	Complete	/api/progress aggregates quiz average, plan count, note count; degrades to zeros if DB disconnected.
Admin Panel	Partial	User listing/stats/role management complete; Announcements and Bulk Clear are intentional stubs.
Deployment	Not deployed	frontend/src/services/api.js still points PRODUCTION_API_BASE at a placeholder domain.

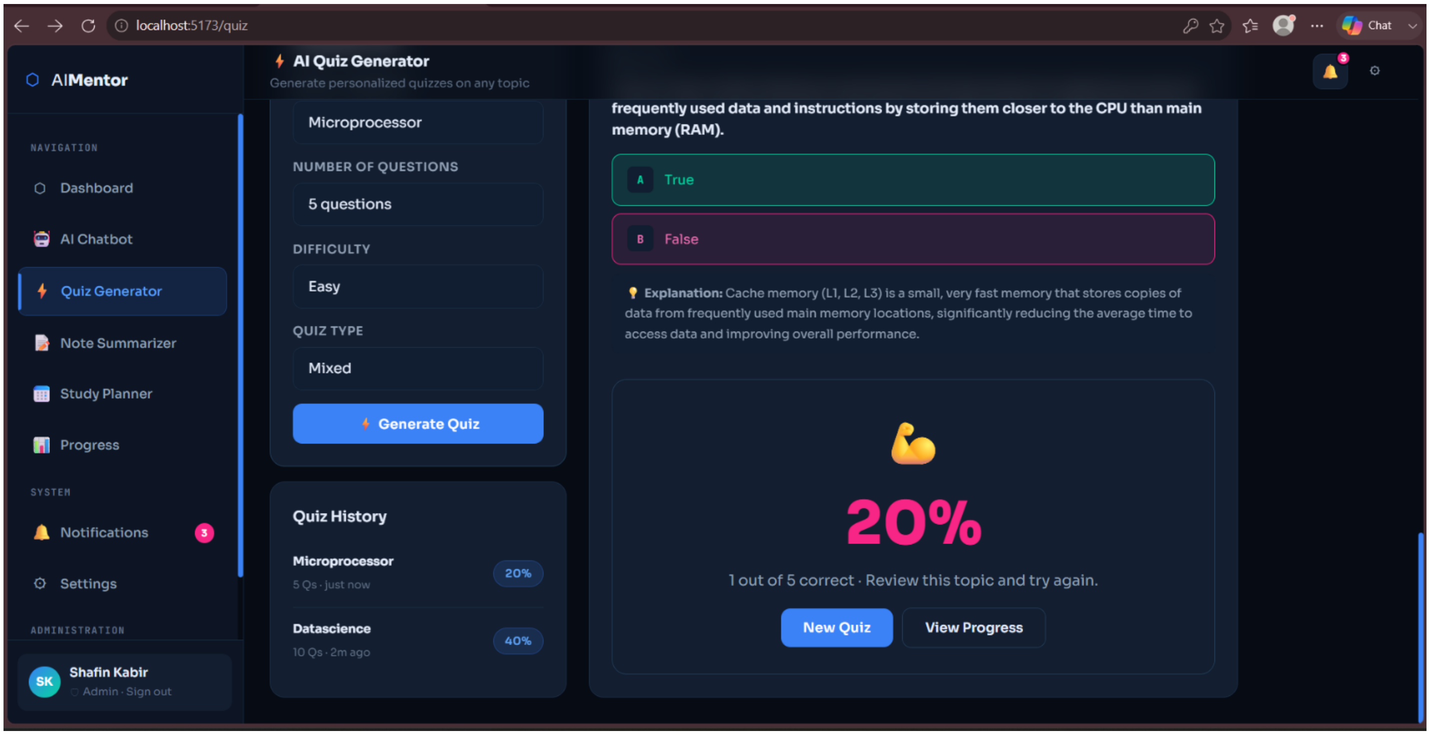
8. Screenshot Gallery

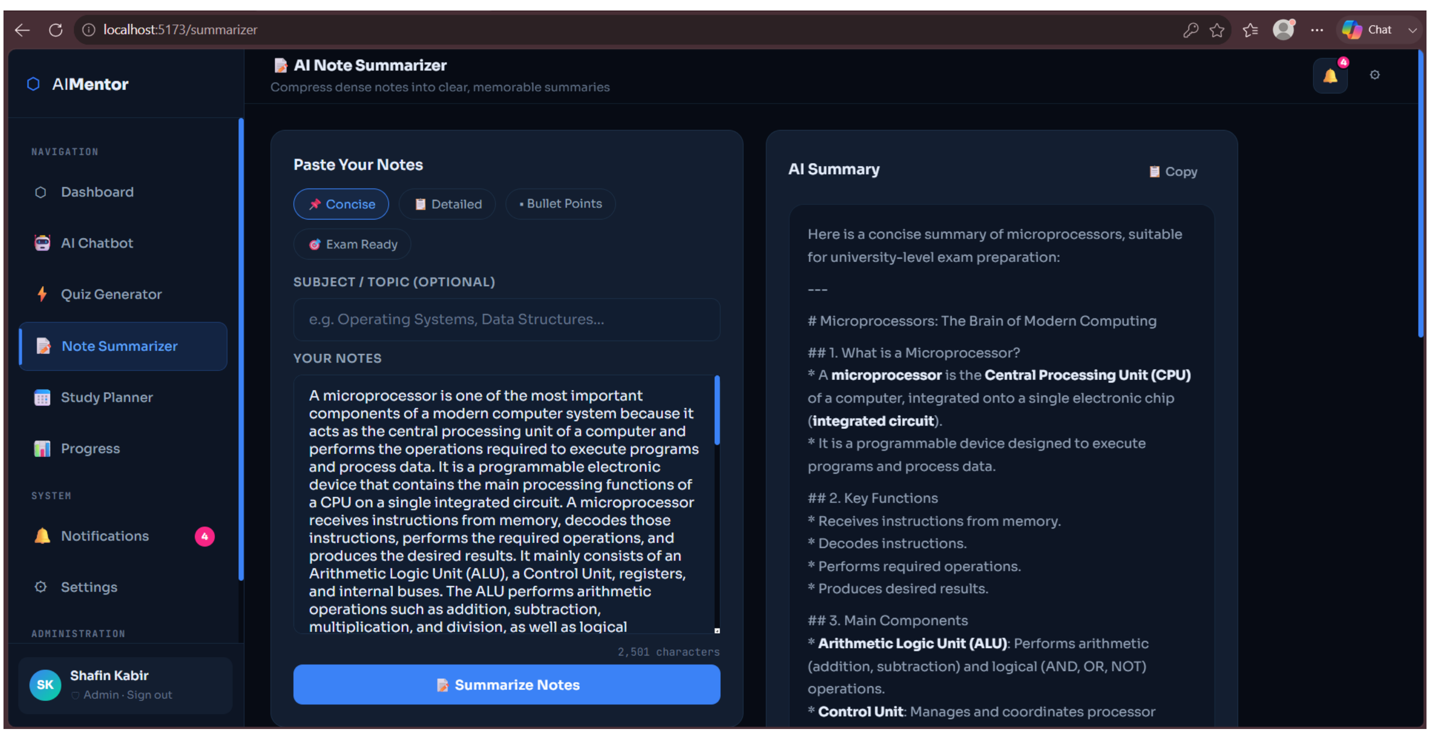
8.1 — AI Feature Interface



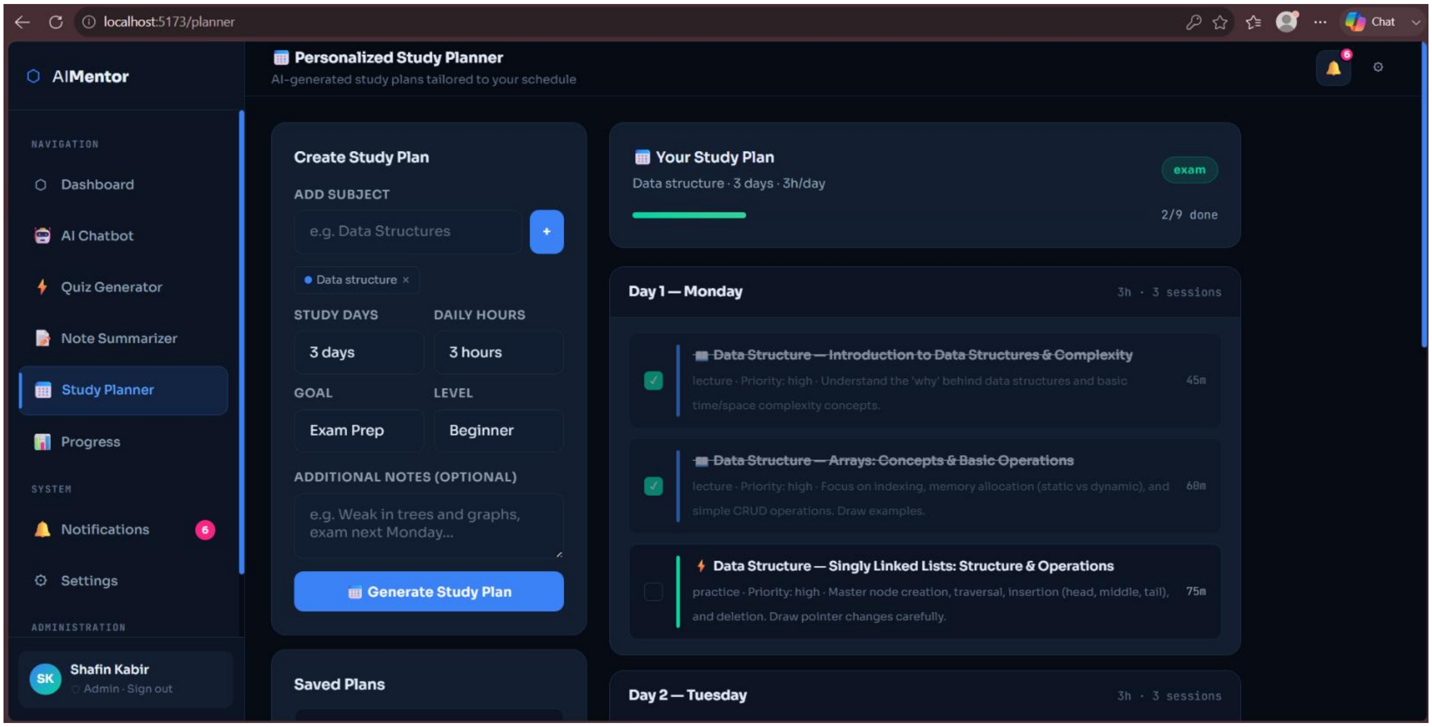
8.2 — AI Feature Interface & AI Responses



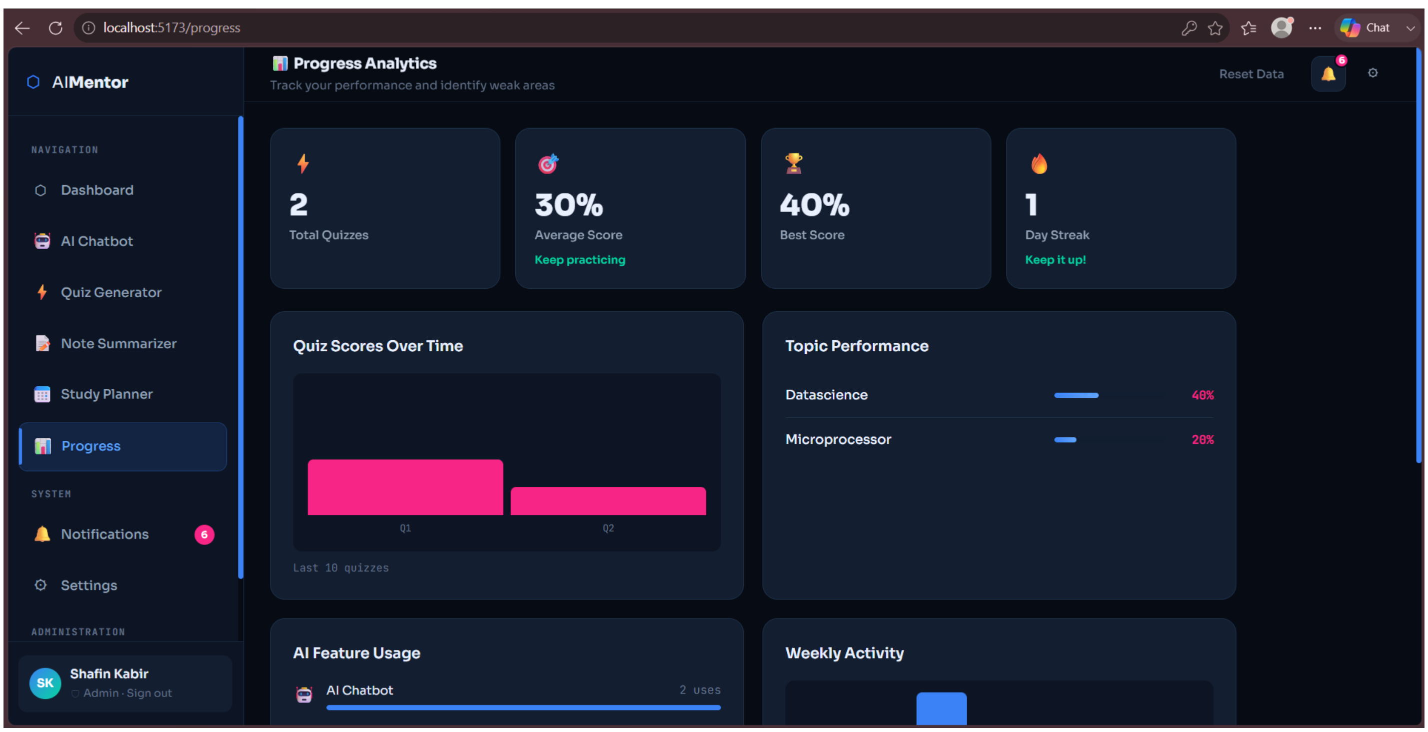

localhost:5173/quiz
AIMentor
AI Quiz Generator
Generate personalized quizzes on any topic
NAVIGATION
Dashboard
AI Chatbot
Quiz Generator
Note Summarizer
Study Planner
Progress
SYSTEM
Notifications
Settings
ADMINISTRATION
Shafin Kabir
Admin · Sign out
Microprocessor
NUMBER OF QUESTIONS
5 questions
DIFFICULTY
Easy
QUIZ TYPE
Mixed
Generate Quiz
Quiz History
Microprocessor
5 Qs · just now
20%
Datascience
10 Qs · 2m ago
40%
frequently used data and instructions by storing them closer to the CPU than main memory (RAM).
A True
B False
Explanation: Cache memory (L1, L2, L3) is a small, very fast memory that stores copies of data from frequently used main memory locations, significantly reducing the average time to access data and improving overall performance.
20%
1 out of 5 correct · Review this topic and try again.
New Quiz
View Progress

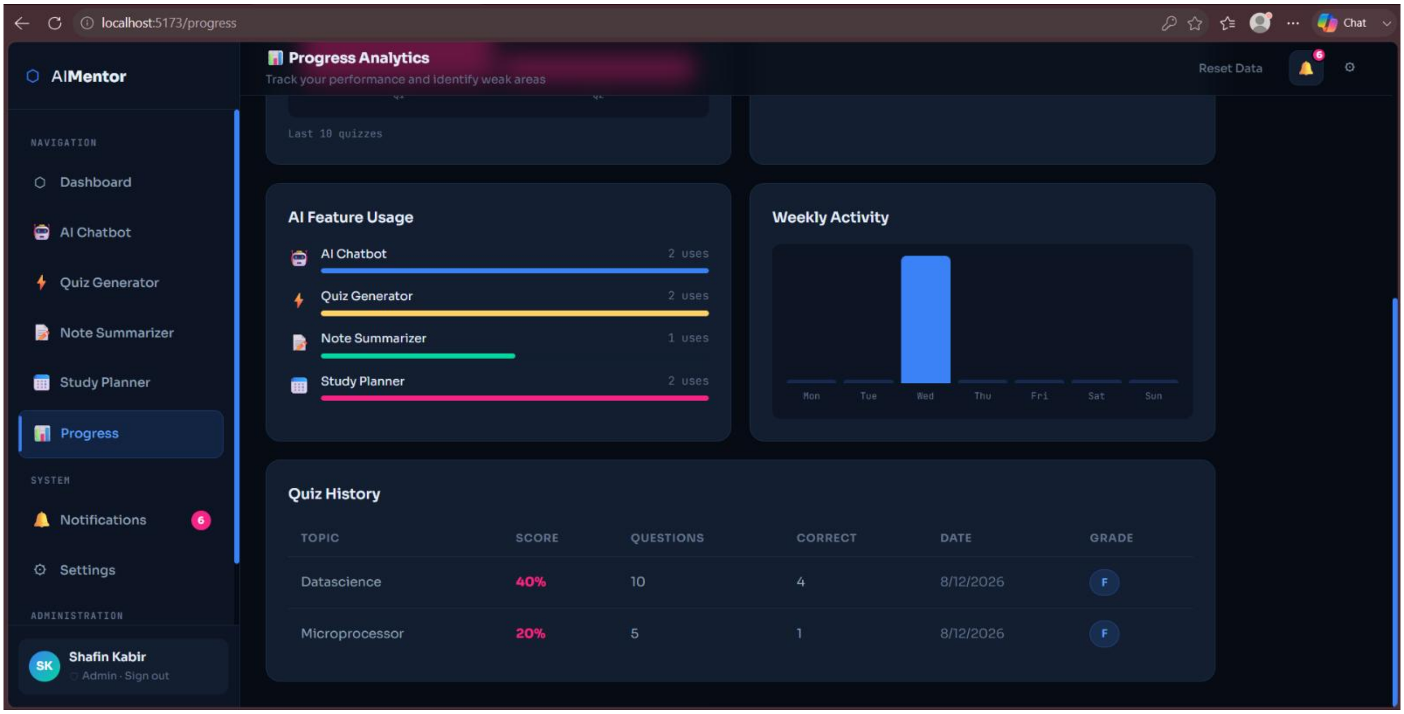

localhost:5173/summarizer
AIMentor
AI Note Summarizer
Compress dense notes into clear, memorable summaries
Paste Your Notes
Concise
Detailed
• Bullet Points
Exam Ready
SUBJECT / TOPIC (OPTIONAL)
e.g. Operating Systems, Data Structures...
YOUR NOTES
A microprocessor is one of the most important components of a modern computer system because it acts as the central processing unit of a computer and performs the operations required to execute programs and process data. It is a programmable electronic device that contains the main processing functions of a CPU on a single integrated circuit. A microprocessor receives instructions from memory, decodes those instructions, performs the required operations, and produces the desired results. It mainly consists of an Arithmetic Logic Unit (ALU), a Control Unit, registers, and internal buses. The ALU performs arithmetic operations such as addition, subtraction, multiplication, and division, as well as logical
2,581 characters
Summarize Notes
AI Summary
Copy
Here is a concise summary of microprocessors, suitable for university-level exam preparation:

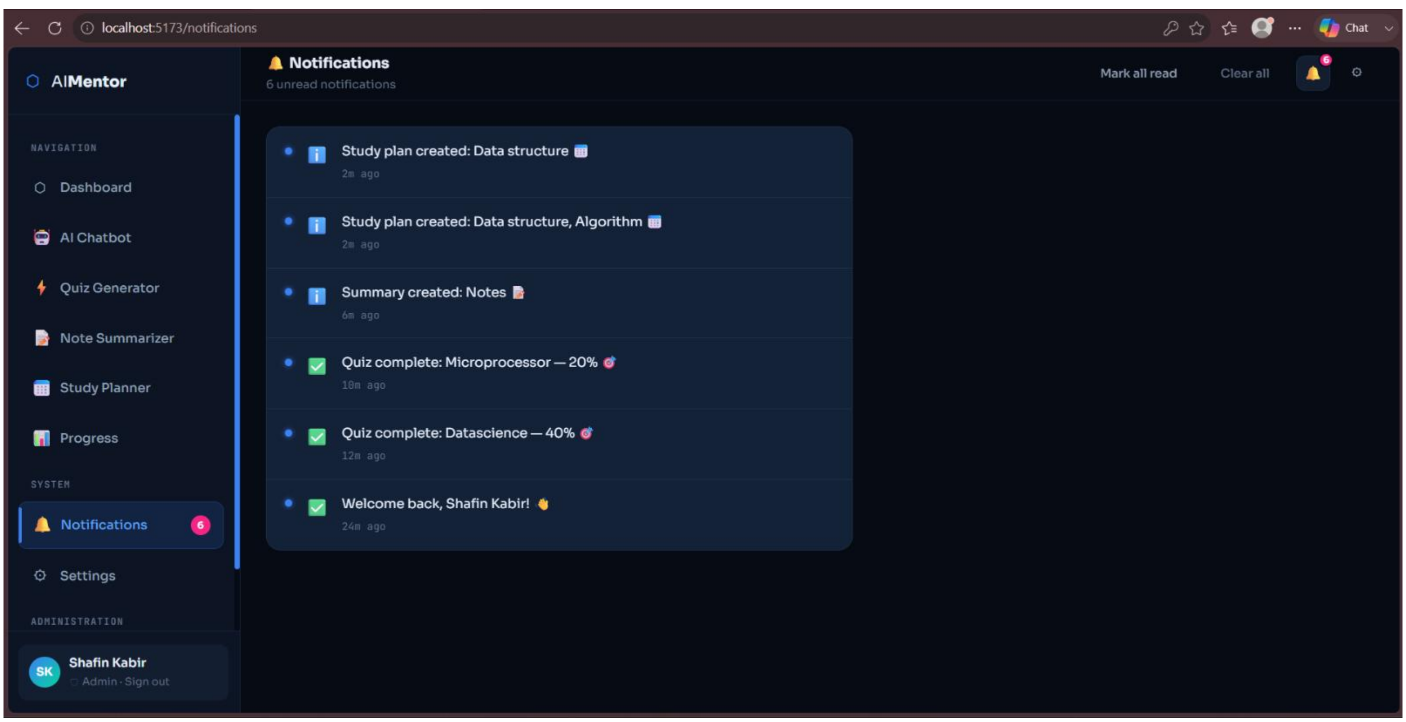
Microprocessors: The Brain of Modern Computing
1. What is a Microprocessor?
* A microprocessor is the Central Processing Unit (CPU) of a computer, integrated onto a single electronic chip (integrated circuit).
* It is a programmable device designed to execute programs and process data.
2. Key Functions
* Receives instructions from memory.
* Decodes instructions.
* Performs required operations.
* Produces desired results.
3. Main Components
* Arithmetic Logic Unit (ALU): Performs arithmetic (addition, subtraction) and logical (AND, OR, NOT) operations.
* Control Unit: Manages and coordinates processor



8.3 — AI Progress




localhost:5173/progress
AIMentor
Progress Analytics
Track your performance and identify weak areas
Reset Data
Last 10 quizzes
NAVIGATION
Dashboard
AI Chatbot
Quiz Generator
Note Summarizer
Study Planner
Progress
SYSTEM
Notifications
Settings
ADMINISTRATION
Shafin Kabir
Admin · Sign out
AI Feature Usage
AI Chatbot 2 uses
Quiz Generator 2 uses
Note Summarizer 1 uses
Study Planner 2 uses
Weekly Activity
Mon Tue Wed Thu Fri Sat Sun
Quiz History
TOPIC SCORE QUESTIONS CORRECT DATE GRADE
Datascience 40% 10 4 8/12/2026 F
Microprocessor 20% 5 1 8/12/2026 F


localhost:5173/notifications
AIMentor
Notifications
6 unread notifications
Mark all read
Clear all
NAVIGATION
Dashboard
AI Chatbot
Quiz Generator
Note Summarizer
Study Planner
Progress
SYSTEM
Notifications
Settings
ADMINISTRATION
Shafin Kabir
Admin · Sign out
Study plan created: Data structure
2m ago
Study plan created: Data structure, Algorithm
2m ago
Summary created: Notes
6m ago
Quiz complete: Microprocessor — 20%
10m ago
Quiz complete: Datascience — 40%
12m ago
Welcome back, Shafin Kabir!
24m ago

8.4 — Prompt Examples

```
const prompt = `Create ${numQuestions} multiple-choice questions on "
  ${topic}" at ${difficulty} difficulty. ` +
  `Return ONLY valid JSON:
  {"questions":[{"question":"...","options":["A","B","C","D"],"answer":"..."}]}.
  No markdown, no extra text.`;
const result = await processAIChat(prompt, 'Act as a quiz generator that outputs strict JSON only.');
const parsed = tryParseJSON(result.text);
```

```
const prompt = `Create a ${days}-day study plan (${hoursPerDay}
    hours/day) covering: ${Array.isArray(subjects) ? subjects.join(', ') : subjects}. ` +
   `Return ONLY valid JSON: {"days":[{"day":1,"sessions":[{"subject":"...","topic":"...",
    "duration_min":60,"type":"lecture"}]}]}. No markdown.`;
const result = await processAIChat(prompt, 'Act as a study planner that outputs strict JSON only.');
const parsed = tryParseJSON(result.text);
```

8.5 — Backend Integration

```
// 1. AI Chat Route
router.post('/chat', softAuth, async (req, res) => {
```

```
// 2. AI Quiz Route
router.post('/quiz', async (req, res) => {
```

```
// 4. AI Study Plan Route
router.post('/plan', async (req, res) => {
```

```
// 3. AI Summary Route
router.post('/summarize', async (req, res) => {
```

AI_Gemini_Model.png

```
  const fullPrompt = systemPrompt
    ? `${systemPrompt}\nUser: ${userMessage}`
    : userMessage;

  if (GEMINI_KEY_LOOKS_VALID) {
    try {
      console.log("🚀 Calling Gemini API...");
      const response = await aiGen.models.generateContent({
        // 'gemini-2.0-flash' was shut down by Google on June 1, 2026.
        // 'gemini-2.5-flash' is the official migration target — good
        // free-tier limits, same "flash" price/speed class.
        // If you hit free-tier rate limits often, try 'gemini-2.5-flash-lite'
        // instead (lower quality, higher RPM/RPD on the free tier).
        model: 'gemini-2.5-flash',
        contents: fullPrompt
      });
```

AI_HuggingFace_model.png

```
  if (process.env.HF_ACCESS_TOKEN) {
    try {
      const hfResponse = await hf.chatCompletion({
  model: "Qwen/Qwen2.5-7B-Instruct",
  messages: [
    {
      role: "user",
      content: fullPrompt
    }
  ],
  max_tokens: 250
});

return {
  source: "HuggingFace Chat",
  text: hfResponse.choices?.[0]?.message?.content || "No response."
};
    } catch (hfErr) {
      console.warn('⚠️ HuggingFace fallback failed:', hfErr.message);
    }
  } else {
    console.warn('⚠️ HF_ACCESS_TOKEN not set — HuggingFace fallback is skipped. Get a free token at https://huggingf
```

AI_Integration.png

```javascript
const express = require('express');
const router = express.Router();
const jwt = require('jsonwebtoken');
const { GoogleGenAI } = require('@google/genai');
const { HfInference } = require('@huggingface/inference');
const { isDbConnected } = require('../middleware/db');


const GEMINI_KEY_LOOKS_VALID =
  typeof process.env.GEMINI_API_KEY === "string" &&
  process.env.GEMINI_API_KEY.trim().length > 0;

if (!GEMINI_KEY_LOOKS_VALID) {
  console.warn("⚠️ GEMINI_API_KEY is missing.");
}

const aiGen = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
const hf = new HfInference(process.env.HF_ACCESS_TOKEN || '');

```

8.6 — GitHub Repository